

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Periphery	Documentation quality	High	
Timeline	2026-02-23 through 2026-03-03	Test quality	High	
Language	Solidity	Total Findings	9	Fixed: 7 Acknowledged: 2
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	None	Medium severity findings ⓘ	3	Fixed: 3
Source Code	https://github.com/VenusProtocol/venus-periphery  #d60851e 	Low severity findings ⓘ	5	Fixed: 3 Acknowledged: 2
Auditors	- Andrei Stefan Auditing Engineer - Leonardo Passos Senior Research Engineer - Paul Clemson Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	1	Fixed: 1

Summary of Findings

The audit for the `RelativePositionManager` and `PositionAccount` contracts within the Venus Protocol identified several critical vulnerabilities, particularly around permission control and user experience. A significant medium-severity issue arises from the function `deactivatePosition()`, where attackers can exploit the permissionless `mintBehalf` function to donate marginal amounts of long-market vTokens to a user's position account, preventing its deactivation. This attack locks user funds, causing significant inconvenience and potentially financial loss for affected users. To mitigate this risk, it is recommended to implement a mechanism that allows `deactivatePosition` to sweep remaining long tokens while permitting closure based on outstanding short debt only.

Another area of concern is the unrestricted `executePositionAccountCall`, which permits governance actors to perform arbitrary calls, including malicious actions that could lock or disable specific user accounts. The absence of stringent access controls here can lead to targeted censorship, undermining user confidence. To enhance security, it is suggested to restrict these calls from modifying critical parameters like the delegate status of position accounts or maintain a whitelist for allowed function calls. Furthermore, the contract's `pause()` functionality risks leaving users vulnerable during market fluctuations, as it restricts their ability to manage positions while allowing liquidations to proceed unchecked. To safeguard users, it is recommended to allow certain operations, like closing positions, even when the contract is paused. Implementing these recommendations would significantly strengthen the resilience and user-friendliness of the contract, increasing overall security.

Fix Review Update: In the recent update, vulnerabilities `VRPM-1`, `VRPM-2`, `VRPM-3`, `VRPM-5`, `VRPM-7`, `VRPM-8`, and suggestions S1, S2, and S3 have been resolved and fixed. However, vulnerabilities `VRPM-4`, `VRPM-6`, and `VRPM-9` have only been acknowledged by the client. In the case of `VRPM-4`, the client explained that the rounding difference is minimal and recoverable, posing no risk of loss. For `VRPM-6`, the client detailed the safeguards in place via the `AccessControlManager` and governance processes to mitigate potential misuse. Lastly, regarding `VRPM-9`, the client reassured that the cap on `DSA` tokens is intentional for market control and deemed adequate. Overall, while most vulnerabilities have been addressed, the acknowledged issues indicate areas that still require monitoring or might not be fully resolved according to standard security practices.

ID	DESCRIPTION	SEVERITY	STATUS
VRPM-1	Permissionless <code>mintBehalf</code> Donation Permanently Blocks <code>deactivatePosition</code> , Locking User Funds	● Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
VRPM-2	Treasury Fee Not Handled in <code>_redeemLongAndSwapToDSA</code> Possible Dos when <code>treasuryPercent > 0</code> and <code>Dsa != Long</code>	• Medium ⓘ	Fixed
VRPM-3	Stale Principal Validation in <code>closeWithLoss</code> when <code>Dsa == Long</code> Due to Treasury Fee Gross-Up	• Medium ⓘ	Fixed
VRPM-4	Rounding Error Accumulation in <code>Dsa == Long</code> Principal Tracking	• Low ⓘ	Acknowledged
VRPM-5	<code>PositionAccount</code> Market Memberships Accumulate Across Cycles and Are Never Exited	• Low ⓘ	Fixed
VRPM-6	<code>executePositionAccountCall</code> Allows Governance to Censor Individual Users by Bricking Their Position Accounts	• Low ⓘ	Acknowledged
VRPM-7	No Migration Path for Existing Position Accounts After <code>setPositionAccountImplementation</code> Change	• Low ⓘ	Fixed
VRPM-8	<code>pause()</code> Blocks User Operations but Venus Liquidations Remain Active, Creating a Defenseless Liquidation Window	• Low ⓘ	Fixed
VRPM-9	<code>dsaVTokenIndexCounter</code> Uses <code>uint8</code> Limiting Dsa Markets to 256	• Informational ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: [https://github.com/VenusProtocol/venus-periphery\(d60851e\)](https://github.com/VenusProtocol/venus-periphery(d60851e))

Repo: <https://github.com/VenusProtocol/venus-periphery>

Included Paths: `contracts/RelativePositionManager`

Extensions: `sol`

Files Excluded

Repo: [https://github.com/VenusProtocol/venus-periphery\(d60851e\)](https://github.com/VenusProtocol/venus-periphery(d60851e)) Files: Not provided

Operational Considerations

1. **PositionAccount Isolation:** Each user position deploys a dedicated EIP-1167 minimal proxy clone with a deterministic address derived from `keccak256(user, longVToken, shortVToken)`. The clone delegates to both RPM and LeverageManager in the Comptroller during initialization. These delegations are permanent — there is no revocation function on the PositionAccount. The only way to revoke is via governance calling `executePositionAccountCall`.
2. **Position Account Reuse Across Cycles:** When a user deactivates and re-activates a position for the same long/short pair, the same PositionAccount is reused. A `cycleId` is incremented to distinguish cycles. Across cycles, old DSA market memberships in the Comptroller are never exited — the position account accumulates market entries that may affect liquidation calculations.
3. **Proportional Close Tolerance:** RPM uses a 2% tolerance band (`PROPORTIONAL_CLOSE_TOLERANCE = 2`) for proportional close operations. This allows users to deviate $\pm 2\%$ from the expected proportional amounts when closing positions. The same constant is used in the max leverage formula (`_getMaxLeverage`), coupling two conceptually independent parameters.
4. **Dust Accumulation on RPM:** The RPM's `_performSwap` and `_redeemLongAndSwapToDSA` functions handle tokens through the RPM contract itself. Any residual tokens from rounding in `mintBehalf` or swap operations accumulate on the RPM permanently. The `_transferDustFromAccountToUser` function only sweeps the PositionAccount, not the RPM itself. There is no `sweep` function on the RPM, so recovery requires a contract upgrade. Consider adding a governance-controlled sweep function.
5. **Treasury Fee Impact:** When `COMPTRROLLER.treasuryPercent()` > 0 , Venus deducts a fee from redeemed underlying tokens. `LeverageStrategiesManager` correctly grosses up the redeem amount, but RPM's `_redeemLongAndSwapToDSA` does not, causing `closeWithProfit` to revert in the DSA != Long case when the treasury fee is active.
6. **Assumed Venus Protocol Security:** The system assumes the security of the other parts of the Venus protocol it interacts with (Comptroller, VToken, ResilientOracle).
7. **Trapped Tokens in PositionAccount:** ERC-20 tokens sent directly to a PositionAccount will likely be trapped permanently without any rescue functionality.
8. **No Governance Takeover Assumed:** No governance takeover is assumed at any point in time.
9. **Secure Upgrade Process:** The upgrade process (timelock, multisig) for the RPM and LeverageStrategiesManager proxies is assumed to be secure.
10. **Oracle Freshness and Accuracy:** The Venus oracle (`COMPTRROLLER.oracle()`) is assumed to always return fresh, accurate prices.
11. **Implementation Set Before First Activation:** `POSITION_ACCOUNT_IMPLEMENTATION` must be set before the first user activation.
12. **Sufficient Market Liquidity:** Venus markets used by positions are assumed to have sufficient cash to service flash loans. Insufficient liquidity causes transient reverts on open/close operations.
13. **No Token Blacklisting:** DSA underlying tokens are assumed not to be blacklisted for the RPM contract or position accounts. This is particularly critical for tokens that are not censorship-resistant, such as USDC.

Key Actors And Their Capabilities

1. **Users (RelativePositionManager):** Can activate isolated leveraged positions with a 3-token model (DSA + Long + Short) via `RelativePositionManager`. Each position deploys a dedicated `PositionAccount` (EIP-1167 clone) that holds the user's funds in isolation. Users can open positions (borrow short, swap to long), close with profit or loss (proportional close with BPS-based fractions), supply or withdraw principal, and deactivate positions. Users interact only with RPM; the RPM orchestrates operations through the PositionAccount and LeverageManager.
2. **Governance (RelativePositionManager):** Controls access via `AccessControlledV8`. Can call `executePositionAccountCall` to execute arbitrary calls from any PositionAccount via `genericCalls`. Can set the PositionAccount implementation (`setPositionAccountImplementation`), manage DSA vToken whitelist (`addDSAVToken`, `setDSAVTokenActive`), and pause/unpause the contract.
3. **PositionAccount (Clone):** Minimal proxy that holds user funds. Delegates to both RelativePositionManager and LeverageManager in the Comptroller during initialization. Forwards `enterLeverage`, `exitLeverage`, and `exitSingleAssetLeverage` calls to LeverageManager. Exposes `genericCalls` (arbitrary external calls) and `transferDustToOwner` — both gated by `onlyRelativePositionManager`.
4. **Privileged Role Attack Vectors:** A compromised governance role can drain any PositionAccount via `executePositionAccountCall` with arbitrary `genericCalls` targets and calldata. There is no timelock, no per-PositionAccount validation, and no target whitelist on this function.

- 5. **LeverageStrategiesManager**: Executes flash-loan-based leverage enter/exit on behalf of position accounts.
- 6. **Venus Comptroller**: Manages markets, collateral factors, flash loans, delegations, and account safety checks. External dependency, not audited here.
- 7. **Venus vTokens**: Hold collateral (long, DSA) and debt (short) for position accounts.
- 8. **Venus Liquidators**: Can liquidate position accounts if health factor drops below a threshold.

Findings

VRPM-1

Permissionless `mintBehalf` Donation Permanently Blocks `deactivatePosition`, Locking User Funds

• Medium ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `1dbb885f6f9c1a3a510a174ef620734dc3d8e94f` .

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: `deactivatePosition` checks `_getLongCollateralBalance(position) > 0` and reverts with `PositionNotFullyClosed` if true. When `longVToken != dsaVToken`, `_getLongCollateralBalance` returns `longVToken.balanceOfUnderlying(positionAccount)`, which includes any externally donated collateral. Since `mintBehalf` is permissionless on Venus vTokens, anyone can call `longVToken.mintBehalf(positionAccount, dustAmount)` to donate a negligible amount (~10 gwei) of the long underlying token to any position account. This inflates the long collateral balance, causing `deactivatePosition` to revert. The attack is repeatable at near-zero cost, permanently blocking deactivation and locking the user's DSA principal inside the position account.

- Exploit Scenario:**
- 1. Alice activates a position with long=CAKE, short=ETH, DSA=USDT (all different tokens)
 - 2. Alice supplies principal and optionally opens/closes a leveraged trade
 - 3. Attacker calls `collateralToken.approve(collateralMarket, 1e10)` then `collateralMarket.mintBehalf(positionAccount, 1e10)` — costs ~10 gwei of the long underlying
 - 4. `deactivatePosition` now reverts with `PositionNotFullyClosed` because `longVToken.balanceOfUnderlying(positionAccount) > 0`
 - 5. Alice's DSA principal remains locked inside the position account with no way to recover it through normal flows
 - 6. Attacker can repeat the donation after any governance rescue at negligible cost

Recommendation: In `deactivatePosition`, instead of reverting when `longCollateral > 0`, only revert on outstanding short debt and sweep any remaining long vTokens (including donations) to the user.

VRPM-2

Treasury Fee Not Handled in `_redeemLongAndSwapToDSA` Possible Dos when `treasuryPercent > 0` and `Dsa != Long`

• Medium ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `8d0c9904e849c9fdfeb5fa93232592263028a5605`, `f3c32abef92d7539fcbd6d1a189a5c0628c1d11d` .

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: In the `DSA != Long` branch of `_redeemLongAndSwapToDSA` at L#882, `longVToken.redeemUnderlyingBehalf(positionAccount, amountToRedeem)` is called. When `COMPTROLLER.treasuryPercent() > 0`, Venus deducts a fee from the redeemed underlying, so RPM receives less than `amountToRedeem`.

Then at L#885-891, `_performSwap(longUnderlying, amountToRedeem, ...)` is called with the full (pre-fee) `amountToRedeem` as `amountIn`. Inside `_performSwap` at L#1042, `tokenIn.safeTransfer(swapHelperAddr, amountIn)` attempts to transfer `amountToRedeem` tokens to SwapHelper. Since RPM holds fewer tokens than `amountToRedeem` (due to the treasury fee), this transfer reverts.

`LeverageStrategiesManager` correctly handles this at L#584-588 by grossing up the redeem amount. The RPM's profit-conversion path lacks this adjustment.

```
// RelativePositionManager.sol L#882-891
uint256 err = longVToken.redeemUnderlyingBehalf(positionAccount, amountToRedeem);
if (err != SUCCESS) revert RedeemBehalfFailed(err);
```



```
uint256 amountOut = _performSwap(
    longUnderlying,
    amountToRedeem,          // uses requested amount, not actual received
    dsaUnderlying,
    minAmountOutProfit,
    swapDataProfit
);
```

Recommendation: Use a balance-before/after pattern to determine the actual received amount

VRPM-3

Stale Principal Validation in `closeWithLoss` when `Dsa == Long` Due to Treasury Fee Gross-Up

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `3753434cf995108d7fb3dd0bdc2b9c004ef730ca`, `6d402e95508f96dc973b315610f2c0ae41d54538`.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: In `closeWithLoss`, the check at L#545-548 validates `dsaAmountToRedeemForSecondSwap` against `_getSuppliedPrincipalBalance(position)` before the first `exitLeverage` call at L#561. When `DSA == Long`, both principal and long collateral share the same vToken pool. This check assumes the principal balance will remain approximately the same after the first leg, but when `COMPTROLLER.treasuryPercent() > 0`, the `LeverageStrategiesManager` grosses up the redeem amount at L#585-587 of `LeverageStrategiesManager.sol`, causing significantly more vTokens to be burned from the shared pool than the RPM's proportional close logic anticipates.

The LM's `_handleExitCollateral` computes `redeemAmount = collateralAmountToRedeem / (1 - treasuryPercent)` and calls `redeemUnderlyingBehalf(PA, redeemAmount)`. The PA's vToken balance decreases by vTokens worth `redeemAmount` (the gross amount), not `collateralAmountToRedeem`. Since `maxLongToWithdraw` in `_getProportionalCloseAmounts` is capped by the net `longBalance`, the first exit can burn up to `longVTokensNet / (1 - fee)` vTokens, exceeding the long collateral portion and eating into principal vTokens.

After the first exit, the actual available principal is lower than what was validated at L#546, but `suppliedPrincipalVTokens` is unchanged. If the user's `dsaAmountToRedeemForSecondSwap` was validated against the pre-exit principal, the second exit can fail at the Comptroller level due to insufficient vTokens.

Recommendation: Move the principal validation after the first exit, or re-sync and re-validate after the first leg when `DSA == Long`.

```
// After the first exitLeverage:
if (longAmountToRedeemForFirstSwap > 0 && address(dsaVToken) == position.longVToken) {
    _syncSuppliedPrincipal(position);
}

// Then validate dsaAmountToRedeemForSecondSwap against updated principal
if (amountToRepaySecond > 0 && dsaAmountToRedeemForSecondSwap > 0) {
    uint256 principalUnderlying = _getSuppliedPrincipalBalance(position);
    if (dsaAmountToRedeemForSecondSwap > principalUnderlying) revert InsufficientWithdrawableAmount();
}
```

VRPM-4

Rounding Error Accumulation in `Dsa == Long` Principal Tracking

• Low ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We acknowledge this finding. The rounding difference is dust-level (at most 1 vToken per operation), remains with the user's position, and is fully recoverable via close or deactivate, so no funds are lost.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: When DSA and long share the same vToken market, `_redeemLongAndSwapToDSA` at L#876-879 estimates `vTokensMinted` using integer division without any actual on-chain mint operation. This truncating division means `suppliedPrincipalVTokens` increases by slightly less than the true vToken equivalent on each call. Over multiple `closeWithProfit` operations on the same position, the accumulated rounding error causes `suppliedPrincipalVTokens` to understate the true principal, which in turn overstates the perceived long collateral balance returned by `_getLongCollateralBalance` at L#1307: `uint256 longVTokensNet = vTokenBalance - position.suppliedPrincipalVTokens`. For a position with many partial close+reopen cycles, the drift accumulates. With Venus's typical exchange rates (~2e26), each rounding error is at most 1 vToken, but over hundreds of operations this can materially affect accounting for high-value positions.

Recommendation: Keep track of the dust value, and each time `vTokensMinted` is calculated, add the over-time accumulated dust to the principal.

VRPM-5

PositionAccount Market Memberships Accumulate Across Cycles and Are Never Exited

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `c30482037c60d1cf481de7b1aa16bbd6cc2dd967`.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: The user's `PositionAccount` enters markets over time (DSA on activation; long via leverage manager; short via borrow path), but normal flows in `RelativePositionManager` never call the `exitMarket()` function. When users rotate different `dsaVToken` choices across cycles, stale memberships persist indefinitely. This creates two different potential issues:

- Account liquidity calculations will include stale markets, meaning positions that appear to be liquidatable based on the actual leverage position from `RelativePositionManager` may not actually be liquidatable if the `PositionAccount` also has collateral from another unrelated market.
- The gas cost of membership checks will increase with every additional market entered, potentially creating a denial of service risk due to transaction gas limits if a large enough number of markets are entered.

Recommendation: When `deactivatePosition` is called, consider calling `dsaVToken.exitMarket()` to ensure that a `PositionAccount` is a member of at most three markets at any one time.

VRPM-6

executePositionAccountCall Allows Governance to Censor Individual Users by Bricking Their Position Accounts

• Low ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

The `executePositionAccountCall` function is restricted via the `AccessControlManager` (ACM), and the corresponding permission is intended to be granted only to governance-related contracts that execute actions through a timelock. Because of this design, any such action would necessarily go through the standard governance process and timelock delay.

This means that even if governance attempted to execute a call such as `updateDelegate`, the action would be publicly visible before execution, and affected users would have sufficient time to withdraw their funds or close their positions during the timelock period. Additionally, this function is intended as an emergency recovery mechanism for governance to resolve edge cases involving position accounts. Given these constraints, selective censorship of users is not expected to occur under normal operating conditions, and the governance + timelock model provides transparency and time for users to react if such an action were ever proposed.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`,
`contracts/RelativePositionManager/PositionAccount.sol`

Description: `executePositionAccountCall` (L219) allows any address with the appropriate `AccessControlManager` role to execute arbitrary calls from any position account via `genericCalls`. While intended as an emergency rescue mechanism, it can be used to revoke the RPM's own delegate status on a targeted position account — calling `COMPTROLLER.updateDelegate(RELATIVE_POSITION_MANAGER, false)` from it — surgically bricking one user's position while the protocol appears normal for everyone else. Unlike `pause()`, which affects all users equally, this allows selective censorship of individual accounts. The user has no self-recovery path since only governance can call `genericCalls`.

Recommendation: Restrict `genericCalls` to disallow calls to the Comptroller's `updateDelegate` function, or maintain an allowlist of permitted target contracts.

VRPM-7

No Migration Path for Existing Position Accounts After `setPositionAccountImplementation` Change

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `cee9803e0dc2e349e5d6f7e0b9faea617bf8f2f6`.

The client provided the following explanation:

The `POSITION_ACCOUNT_IMPLEMENTATION` address is not intended to be modified after the initial deployment.

The setter was introduced solely to resolve the circular dependency between the Position Account (PA) and the RelativePositionManager (RPM): The PA requires the RPM address during initialization. The RPM deploys PAs via deterministic cloning. To break this dependency cleanly, we allow the RPM address to be set once on the PA implementation contract.

This avoids having to initialize or configure the RPM address individually on each deployed proxy, thereby reducing deployment gas costs. A one-time lock has been added to the setter to ensure it cannot be modified after being set.

We do not expect the implementation address to change going forward. Any future functionality can be handled through generic call routing or wrapper logic within the RPM, eliminating the need to upgrade the PA contract itself.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`,
`contracts/RelativePositionManager/PositionAccount.sol`

Description: `setPositionAccountImplementation` (L140) updates the implementation used for future EIP-1167 clones, but already-deployed position accounts are immutable minimal proxies pointing to the old implementation. If a security fix or feature change is needed in `PositionAccount`, existing accounts cannot be upgraded — they permanently run the old logic. Since `_deployPositionAccount` only runs on first activation (when `position.positionAccount == address(0)`), and position accounts persist across deactivation/reactivation cycles, there is no way to force an existing user onto a new implementation. The only workaround is `executePositionAccountCall`, but that cannot change the proxy's implementation pointer — EIP-1167 clones have hardcoded delegation targets. This means a critical bug in `PositionAccount` (e.g., in `enterLeverage`, `exitLeverage`, or `genericCalls`) would be unpatchable for all existing positions.

Recommendation: Consider using UUPS or beacon proxies instead of EIP-1167 clones for position accounts, allowing governance to upgrade all existing accounts in a single transaction.

VRPM-8

`pause()` Blocks User Operations but Venus Liquidations Remain Active, Creating a Defenseless Liquidation Window

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `e312e19903a9f0bc86ba147cf6ee39aade7e5a88`.

The client provided the following explanation:

We have updated the pause mechanism to introduce two levels of pause. Defensive operations such as `closeWithProfit`, `closeWithLoss`, and `supplyPrincipal` remain available during partial pauses so users can protect their positions.

These functions would only be blocked under a full pause, which is intended for critical scenarios and would be executed by governance with prior notice whenever possible, primarily to protect users during emergency fixes.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: All user-facing state-changing functions — `openPosition`, `closeWithProfit`, `closeWithLoss`, `withdrawPrincipal`, `supplyPrincipal`, `deactivatePosition`, and `activatePosition` — are gated by `whenNotPaused`. However, Venus liquidation operates at the Comptroller/vToken level and is entirely unaffected by the RPM's pause state. When governance pauses the RPM, users with open leveraged positions cannot:

- Close their positions to avoid liquidation
- Add collateral (`supplyPrincipal`) to improve their health factor
- Withdraw principal to re-supply elsewhere.

Meanwhile, if market prices move against them during the pause, their position accounts become liquidatable by Venus liquidators. The user is completely defenseless — they can see their position approaching liquidation but have no way to intervene through the RPM.

Recommendation: Allow `closeWithProfit`, `closeWithLoss`, and `supplyPrincipal` to remain callable even when paused, so users can defend their positions against liquidation. Only gate position-opening and activation functions with `whenNotPaused`.

VRPM-9

`dsvTokenIndexCounter` Uses `uint8` Limiting Dsa Markets to 256

• Informational ⓘ

Fixed

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

DSA tokens are intended to represent stablecoin assets, so the number of supported markets will remain very limited and controlled. Therefore, the uint8 limit of 256 markets is considered more than sufficient for the intended use case.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: `dsaVTokenIndexCounter` at L#60 is declared as `uint8`, which limits the total number of DSA vToken markets that can ever be added to 256. Solidity 0.8.28 reverts on overflow, so attempting to add the 257th DSA market will permanently brick the `addDSAVToken` function. Deactivated DSAs still consume their index, there is no mechanism to reclaim indices.

Recommendation: Use `uint256` for future-proofing.

Auditor Suggestions

S1 `POSITION_ACCOUNT_IMPLEMENTATION` Named as Immutable but Is Mutable

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `cee9803e0dc2e349e5d6f7e0b9faea617bf8f2f6`.
The client provided the following explanation:

We acknowledge this. A one-time lock has been added to the setter, ensuring the value can only be set once. This makes the variable effectively immutable after initialization, clarifying the intended behavior.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: The variable `POSITION_ACCOUNT_IMPLEMENTATION` at L#57 uses `SCREAMING_SNAKE_CASE` (Solidity convention for constants and immutables) but is actually a regular mutable state variable with a governance setter `setPositionAccountImplementation` at L#131. This naming is misleading and may cause auditors or integrators to assume the value cannot change.

Recommendation: Rename to `positionAccountImplementation` to match Solidity naming conventions for mutable state variables.

S2 `_performSwap` Ignores Revert Reason From `Swaphelper`

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `82eaa9e87247932a7e1c3f67c6b5efffd9633ee8`.

File(s) affected: `contracts/RelativePositionManager/RelativePositionManager.sol`

Description: RPM's `_performSwap` at L#1046 uses a low-level `.call()` to the swap helper and discards the return data on failure. If the `SwapHelper` reverts with a specific error (e.g., `DeadlineReached`, `Unauthorized`, `SaltAlreadyUsed`), the revert reason is discarded and replaced with the generic `TokenSwapCallFailed`. This makes debugging and user-facing error messages significantly worse. In contrast, `PositionAccount.genericCalls` correctly uses `AddressUpgradeable.functionCall` which propagates the revert reason.

Recommendation: Use a clear revert reason not a generic one

S3 Flash loan fees silently increase short debt beyond user-requested borrow amount

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `923d08baf9dd809d25a004ccdfa6bad190c7845`.

File(s) affected: contracts/LeverageManager/LeverageStrategiesManager.sol

Description: When `openPosition` executes, the `PositionAccount` calls `LeverageStrategiesManager.enterLeverage`, which takes a flash loan of the short asset. Inside the flash loan callback (`_handleEnterCollateral`), the fee is repaid by calling `borrowBehalf(onBehalf, borrowedAssetFees)` — borrowing the premium on behalf of the position account and immediately repaying it to the flash loan. This means the position's short debt is `shortAmount + flashLoanFee`, not just `shortAmount`. Users who check their borrow amount after opening will see a higher debt than expected, with no on-chain event or return value distinguishing the user-requested borrow from the fee component. This can cause confusion when calculating close parameters.

Recommendation: Document flash loan fee behavior in public-facing user documentation and UI. Consider emitting the fee amount as part of the `PositionOpened` event so users and off-chain tooling can distinguish the requested borrow from the fee overhead.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/VenusProtocol/venus-periphery`
- `a75...bfc ./contracts/RelativePositionManager/IPositionAccount.sol`
 - `ada...a64 ./contracts/RelativePositionManager/IRelativePositionManager.sol`
 - `60a...976 ./contracts/RelativePositionManager/PositionAccount.sol`
 - `574...d90 ./contracts/RelativePositionManager/RelativePositionManager.sol`

Test Suite Results

`npx hardhat test tests/hardhat/RelativePositionManager/RelativePositionManager.ts`

```
RelativePositionManager
  Deployment & Initialization
Warning: Potentially unsafe deployment of
contracts/RelativePositionManager/RelativePositionManager.sol:RelativePositionManager

You are using the `unsafeAllow.state-variable-immutable` flag.

  ✓ should expose correct immutables via proxy
  ✓ should revert when implementation is deployed with zero address for any constructor parameter
Warning: Potentially unsafe deployment of
```

You are **using** the ``unsafeAllow.state-variable-immutable`` flag.

- ✓ should revert **when** PositionAccount implementation **is not set before usage**
- ✓ should **update** PositionAccount implementation via governance-controlled setter

pause

- ✓ should block state-changing **user** operations **when** paused
- ✓ should allow activation again **after** unpause

addDSAVToken

- ✓ should **add** DSA vToken **and** emit event
- ✓ should revert **when** adding zero address
- ✓ should revert **when** adding unlisted market
- ✓ should revert **when** caller **is not** allowed **by** ACM
- ✓ should revert **when** adding vBNB market (VBNBNotSupported)

setDSAVTokenActive

- ✓ should allow **using** DSA **when** active, block **when** disabled, **and** allow again **when** re-enabled (47ms)

activatePosition

- ✓ should revert **when** longVToken **is** zero
- ✓ should revert **when** shortVToken **is** zero
- ✓ should revert **when** market **is not** listed
- ✓ should revert **when** longVToken **is** vBNB (VBNBNotSupported)
- ✓ should revert **when** shortVToken **is** vBNB (VBNBNotSupported)
- ✓ should revert **when** effective leverage **is** below minimum
- ✓ should revert **when** effective leverage **is** above maximum
- ✓ should activate position **and** deploy position account
- ✓ should revert **when** activating the same position again
- ✓ should activate **with** initial principal **when** user approves **and** supplies
- ✓ should reuse existing position account **when** reactivating a fully closed position (55ms)

activateAndOpenPosition

- ✓ should activate **and open in one call** (119ms)

supplyPrincipal

- ✓ should revert **when** amount **is** zero
- ✓ should revert **when** position **is not** active
- ✓ should increase principal **and** emit event

executePositionAccountCall

- ✓ should revert **when** caller **is not** allowed **by** ACM
- ✓ should succeed **when** caller **is** allowed **and** emit GenericCallExecuted

openPosition

- ✓ should revert **when** position **is not** active
- ✓ should revert **when** short amount **is** zero
- ✓ should revert **when no** principal **and no** additional principal
- ✓ should **open** position successfully **when** swap data sweeps long **to** LM (105ms)

closeWithProfit

- ✓ closeWithProfit: should revert **when** position **is not** active
- ✓ closeWithProfit: 50% BPS with repay 0 and redeem 0 (all 50% for profit) should revert **as** debt

must be repaid (121ms)

- ✓ closeWithProfit (partial 50%, no profit): closed at same price, should reduce debt **and** long

proportionally (161ms)

- ✓ closeWithProfit (90%): **close 90% with** profit swap (179ms)
- ✓ closeWithProfit (100%): **full** detailed – exact swap behaviour **and all** transfers (217ms)
- ✓ closeWithProfit: **when no** debt but long available, 100% **close** redeems **full** long **as** profit **and**

retains it **as** principal (169ms)

closeWithLoss

- ✓ closeWithLoss: should revert **when** position **is not** active
- ✓ closeWithLoss: should revert **when** there **is no** debt (ZeroDebt) (122ms)
- ✓ closeWithLoss (partial 95%): should repay 95% of debt **and** redeem 95% of long; 5% remains (220ms)
- ✓ closeWithLoss: repay **with** DSA **only** **when** there **is no** long remaining (first **exit** skipped) (172ms)
- ✓ closeWithLoss (100%): one swap – long covers repay proportionally; debt **and** long go **to** zero

(163ms)

DSA market used **as** long market (USDT): longVToken == dsaMarket

- ✓ openPosition: should increase suppliedPrincipal **when** additionalPrincipal **is** provided (122ms)
- ✓ closeWithProfit: should **close** fully **and** increase suppliedPrincipal vTokens (profit realized **in**

same underlying) (176ms)

- ✓ closeWithLoss: should **close** fully; second **exit** uses same market **as** DSA/long **and** reduces principal

vTokens (235ms)

getUtilizationInfo **and** calculateMaxBorrow

- ✓ should **return** utilization **info for** active position **with** principal (43ms)
- ✓ should **return** max borrow **for** active position **with** principal (41ms)
- ✓ should **return** utilization **with** different oracle prices (DSA **and** short) (77ms)
- ✓ should **return** lower available capital **and** max borrow **when** position has **open** borrow (133ms)
- ✓ should **return** zero available capital **and** max borrow **when** position has **no** principal (38ms)

```
    ✓ should show 0 available for borrow after scaling position to max (use full available capacity) (196ms)
  withdrawPrincipal
    ✓ should revert when position is active and amount exceeds withdrawable (39ms)
    ✓ should withdraw principal when position is active and amount is withdrawable (60ms)
    ✓ should remain usable and allow withdrawing extra principal minted via mintBehalf (92ms)
  deactivatePosition
    ✓ should revert when position is not active
    ✓ should revert when position is active but not fully closed (110ms)
    ✓ should succeed when position is active with principal but no open collateral or debt (65ms)
  DSA change on reactivation
    ✓ should allow reactivation with new DSA after full close and explicit deactivation

60 passing (6s)
```

Code Coverage

npm run hardhat coverage --testfiles "tests/hardhat/RelativePositionManager/RelativePositionManager.ts"

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	100	100	100	100	
Interfaces.sol	100	100	100	100	
contracts/LeverageManager/	57.49	35.58	68.18	50.85	
ILeverageStrategiesManager.sol	100	100	100	100	
LeverageStrategiesManager.sol	57.49	35.58	68.18	50.85	... 776,781,818
contracts/PositionSwapper/	0	0	0	0	
ISwapHelper.sol	100	100	100	100	
PositionSwapper.sol	0	0	0	0	... 432,442,443
WBNBSwapHelper.sol	0	0	0	0	... 133,134,136
contracts/RelativePositionManager/	97.33	66	96.43	93.46	
IPositionAccount.sol	100	100	100	100	
IRelativePositionManager.sol	100	100	100	100	
PositionAccount.sol	90.48	34.38	87.5	81.25	... 242,243,275
RelativePositionManager.sol	97.85	70.64	97.92	94.57	... 5,1256,1336
contracts/SwapHelper/	85.19	38.24	75	64.29	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
SwapHelper.sol	85.19	38.24	75	64.29	... 231,234,235
All files	67.27	42.16	72.12	63.27	

Changelog

- 2026-03-03 - Initial report
- 2026-03-10 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, PolyMarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation

provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

